ECS 240 Project Proposal

Rajiv Choudhury, Jacob Hamzawi, Rishika Nigam, Gurpreet Singh

November 13, 2026

1 Introduction and Motivation

1.1 Problem Domain and Motivation

A *side effect* is a modification of state that outlives a function call, that is, state the function did not create itself. Side effects make code difficult to reason about: at a call site it is unclear which components a function may modify, so a reader must inspect the callee and everything it calls[1]. Functional programming avoids this problem and allows for greater modularity[2], and Backus argued for liberating programming from assignment and mutation altogether[3]. Yet effects are often necessary, as with file I/O. Haskell confines them to the monad[4], and effect systems track them statically in types[5,6]. Our domain is *making side effects explicit in an imperative systems language*: every function is pure unless declared `impure`, an impure function must list the outside state it may modify, and the compiler checks the declaration.

1.2 Proposed Language

We propose a toy programming language similar to traditional systems programming languages like C and Rust with the distinction that every function is explicitly defined as "pure" or "impure", where a "pure" function may only modify state initialized in or moved into its scope. Our language will implement at least the minimum set of elements that allow us to illustrate this system, including variables, functions, references, and an 8-bit unsigned integer primitive as the sole type. We may support other features such as structs, strings, or simple file I/O depending on our timeline (see §§ 2 and 4).

Rust itself originally sought such a design, defining all functions as "pure" by default and employing the `io` keyword to explicitly designate a function with I/O side effects.[7] The language designers eventually opted to remove `io` keyword and the pure/impure separation. We hope to discover while designing and analyzing our own language what explicit declaration of side effects could enable for language users and whether the Rust developers' decision was ultimately justified.

```
fn pure_function(x, y) {
    x + y
}

// global variable
var z;

// You must specify the data out of scope whose
// state may change
fn impure(z) impure_function(x, y) {
    z = x + y;
}

// If variables have fields, those fields
// must be declared
fn impure(x.value, y.value) impure_function_struct(x, y) {
    // mutates x
    x.value = 1;
    y.value = 1;
}

// You can look at the program see a full list
// of all the things that could change state
fn impure main() {
    var x = 1;
    var y = 2;
    x = pure_function(x, y);
          // mutated: z
          impure_function(x, y);
}
```

1.3 Proposed Tool

compiler, where the input is a string in our source language and the output is a string in Rust. The user interacts with it from the command line As a stretch goal (see §2), an analysis tool to glean information from impure declarations may be in scope.

1.4 Scope and Related Work

Haskell has the Monad[4] Functional programming allows for greater modularity.[2] And they also often discourage side effects by the nature of their design. `cargo-scan` is a static analysis tool for Rust that detects side effects from Rust source code.[8]

2 Goals

Minimum goal. Parser, interpreter, and working compiler. Interpreter and compiler must detect whether a function is pure or impure and error if a function is impure without declaring so along with the outside state it modifies.

Target goal. The above, along with a static analysis tool that enumerates all the possible state that is modified given a function (including the functions it itself calls). The tool will at least be a CLI, but could be extended to a VSCode extension as well.

Reach goal. The above, as well as other Rust features that might make the division harder and mitigation: system state (file I/O), structs, etc.? Specifically for file I/O, we could explore options to avoid Rust's issue of `io` keyword explosion (similar to Haskell's Monad).

3 Syntax and Semantics

Syntax. The syntax of our language can be described by the following example and the subsequent draft formal grammar:

```
fn pure_function(x, y) {
    x + y
}

// global variable
var z;

// You must specify the data out of scope whose
// state may change
fn impure(z) impure_function(x, y) {
    z = x + y;
}

// If variables have fields, those fields
// must be declared
fn impure(x.value, y.value) impure_function_struct(x, y) {
    // mutates x
    x.value = 1;
    y.value = 1;
}

// Data reached through a reference is out of scope,
// so writing through it must be declared
fn impure(*r) increment(&r) {
    *r = *r + 1;
}

// You can look at the program see a full list
// of all the things that could change state
fn impure main() {
    var x = 1;
    var y = 2;
    x = pure_function(x, y);
    // mutated: z
    impure_function(x, y);
    // mutated: x (through the reference)
    increment(&x);
}
```

Start symbol P (program). Metavariables: n ranges over integer literals 0–255, x, f, ℓ range over identifiers (letters, digits, and underscores, beginning

with a letter) applying to variable, function, and field names respectively.

$$
\begin{aligned}
P &::= D \mid D\ P \\
D &::= \texttt{var}\ x\texttt{;} \mid \texttt{fn}\ a\ f\texttt{(}q_1, \ldots, q_k\texttt{)}\ \texttt{\{}\ B\ \texttt{\}} \\
q &::= x \mid \texttt{\&}x \\
a &::= \epsilon \mid \texttt{impure} \mid \texttt{impure(}p_1, \ldots, p_m\texttt{)} \\
B &::= s_1 \ldots s_j \mid s_1 \ldots s_j\ e \\
s &::= \texttt{var}\ x\texttt{;} \mid \texttt{var}\ x\ \texttt{=}\ e\texttt{;} \mid p\ \texttt{=}\ e\texttt{;} \mid f\texttt{(}e_1, \ldots, e_k\texttt{);} \\
p &::= x \mid p\texttt{.}\ell \mid \texttt{*}p \\
e &::= n \mid p \mid \texttt{\&}p \mid e_1 \oplus e_2 \mid \texttt{(}e\texttt{)} \mid f\texttt{(}e_1, \ldots, e_k\texttt{)} \\
\oplus &::= \texttt{+} \mid \texttt{-} \mid \texttt{*} \mid \texttt{/}
\end{aligned}
$$

where $k, m, j \geq 0$. A place p names a location that can be read, assigned, or declared in an `impure` clause: a variable, a field of a place, or the target of a reference. `&`p creates a reference to p; all references are mutable (Rust's `&mut`) but are written `&`. Function signatures show which arguments are passed by reference with an explicit symbol `&`x. The grammar for e is ambiguous as written: unary `*` and `&` bind tightest, then `*` and `/`, then `+` and `-`, and all binary operators are left-associative. A trailing expression e with no semicolon is the function's return value, as in Rust.

Semantics. The semantics of our language can be described as the set of `(INPUT, prog, OUTPUT)` triples produced by our reference interpreter, which serves as the oracle for the compiler: for every program, the emitted Rust must produce the same output. Programs take no input, so `INPUT` is always empty. This could change if we decide to investigate file I/O or read from stdin. As in Rust, a program that fails the purity check is rejected before it runs and so belongs to no triple. For every other program, `OUTPUT` is the final values of all global variables once `main` returns, or an error if execution faults (e.g., `u8` overflow or division by zero). Our language's purity check must guarantee that in any accepted program, a call to a function declared `impure(`$p_1, \ldots, p_m$`)` changes only $p_1, \ldots, p_m$, and a call to a pure function cannot change state outside its own scope. The interpreter can verify this by comparing state before and after each call, so any violation exposes a bug in the purity check itself. Should our purity check extend to system state (see

§2), its guarantees would extend along with it (e.g. no file is read or written unless explicitly declared, etc.).

4 Implementation Plan

4.1 Tool and Language Selection

Parser in Rust; meta language is Rust; target language for compiler is Rust.

4.2 Incorporating Existing Tools

From scratch.

4.3 Architecture

Core language definition; parser that outputs core; interpreter reads core and executes; compiler reads core and outputs IR, in IR we do checks on purity, then IR to target Rust

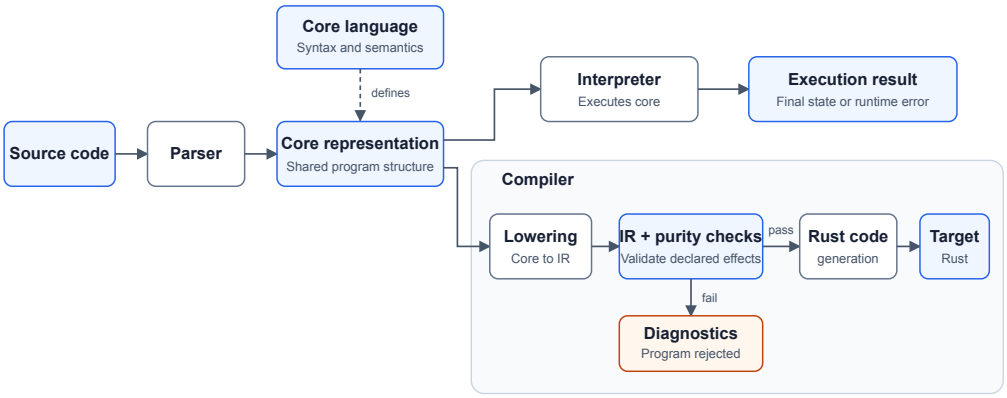


Figure 1: Tool architecture, showing the interpreter and compiler paths from the shared core representation.

4.4 Development Effort

4.5 Challenges

Tracking scope is hard; tool is too much work... Fallback..? What if a variable will *never* change state? if we have if statements...should we detect that? need it be declared?

5 Collaboration Plan

Division of Work. Work divided as follows

Component	Group Member(s)
Core	Jacob Hamzawi
Parser	?
Interpreter	?
Compiler	?
Analysis tool	?

Communication. Slack and GitHub; Meetings after class on Monday and over Zoom on Thursday

Timeline. The below timeline assumes a 4-week time allotment to complete the project. It may be contracted or expanded depending on our eventual project due date.

Week	Components
Week 1	Parser
Week 2	Interpreter + Compiler
Week 3	Analysis tool + exploration
Week 4	Presentation

Commitment statement.

- I commit to work and contribute equally to this project. — *Rajiv Choudhury*
- I commit to work and contribute equally to this project. — *Jacob Hamzawi*

6 Conclusion and Questions for Feedback

Our goal is to design a systems-style language in which functions explicitly declare the outside state they may modify, allowing us to explore whether compiler-checked effect declarations make imperative programs easier to reason about. Developing this language will require tracking scope, identifying mutations through references, and accounting for effects across function calls. A central question is whether tracking mutations to variables and referenced data is enough to evaluate the design, or whether we should also model system state, such as file I/O. We would appreciate feedback on whether the broader scope would justify its added complexity within our timeline.

We will develop a parser, a reference interpreter, and a compiler that translates our language into Rust, rejecting programs whose mutations exceed their declared effects. Our target also includes a command-line static analysis tool that enumerates the state a function may modify, including effects of the functions it calls. Together, these tools will let us examine both the guarantees that explicit effect declarations provide and the annotation effort they require.

References Cited

[1] Alexandru Sălcianu and Martin Rinard. Purity and Side Effect Analysis for Java Programs. In Radhia Cousot, David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi, and Gerhard Weikum, editors, *Verification, Model Checking, and Abstract Interpretation*, volume 3385 of *Lecture Notes in Computer Science*, pages 199–215. Springer Berlin Heidelberg, Berlin, Heidelberg, 2005. ISBN 978-3-540-24297-0 978-3-540-30579-8. doi: 10.1007/978-3-540-30579-8_14. URL http://link.springer.com/10.1007/978-3-540-30579-8_14.

[2] J. Hughes. Why Functional Programming Matters. *The Computer Journal*, 32(2):98–107, February 1989. ISSN 0010-4620, 1460-2067. doi: 10.1093/comjnl/32.2.98. URL https://academic.oup.com/comjnl/article-lookup/doi/10.1093/comjnl/32.2.98.

[3] John Backus. Can programming be liberated from the von Neumann

style? a functional style and its algebra of programs. *Communications of the ACM*, 21(8):613–641, August 1978. ISSN 0001-0782. doi: 10.1145/359576.359579. URL https://dl.acm.org/doi/10.1145/359576.359579.

[4] Simon L. Peyton Jones and Philip Wadler. Imperative functional programming. In *Proceedings of the 20th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, POPL '93, pages 71–84, New York, NY, USA, March 1993. Association for Computing Machinery. ISBN 978-0-89791-560-1. doi: 10.1145/158511.158524. URL https://dl.acm.org/doi/10.1145/158511.158524.

[5] J. M. Lucassen and D. K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, POPL '88, pages 47–57, New York, NY, USA, January 1988. Association for Computing Machinery. ISBN 978-0-89791-252-5. doi: 10.1145/73560.73564. URL https://dl.acm.org/doi/10.1145/73560.73564.

[6] J. P. Talpin and P. Jouvelot. The Type and Effect Discipline. *Information and Computation*, 111(2):245–296, June 1994. ISSN 0890-5401. doi: 10.1006/inco.1994.1046. URL https://www.sciencedirect.com/science/article/pii/S0890540184710467.

[7] Graydon Hoare. Re: Sub-grammar for range pattern constants? rust-dev mailing list, Mozilla, April 2013. URL https://mail-archive.com/rust-dev@mozilla.org/msg03913.html. Accessed: 2026-11-11.

[8] Lydia Zoghbi, David Thien, Ranjit Jhala, Deian Stefan, and Caleb Stanford. Auditing Rust Crates Effectively. In *Programming Languages and Systems: 35th European Symposium on Programming, ESOP 2026, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2026, Turin, Italy, April 11–16, 2026, Proceedings, Part II*, pages 424–444, Berlin, Heidelberg, April 2026. Springer-Verlag. ISBN 978-3-032-22722-5. doi: 10.1007/978-3-032-22723-2_15. URL https://doi.org/10.1007/978-3-032-22723-2_15.

A Complete Formal Grammar (In Progress)

$$
\begin{aligned}
S &::= \\
\texttt{<Int>} &::= 0 \mid \texttt{<NonZeroInt>} \\
\texttt{<NonZeroInt>} &::= \texttt{<NonZeroDigit>} \mid \texttt{<NonZeroInt>} \mathbin{+\!\!+} \texttt{<Digit>} \\
\texttt{<NonZeroDigit>} &::= 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\
\texttt{<Digit>} &::= 0 \mid \texttt{<NonZeroDigit>} \\
\texttt{<Exp>} &::= \texttt{<MultDivExp>} \mid \texttt{<Exp> <AddSubOp> <MultDivExp>} \\
\texttt{<AddSubOp>} &::= + \mid - \\
\texttt{<MultDivExp>} &::= \texttt{<Factor>} \mid \texttt{<MultDivExp> <MultDivOp> <Factor>} \\
\texttt{<Factor>} &::= \texttt{<Int>} \mid \texttt{<Var>} \mid \texttt{(<Exp>)} \\
\texttt{<MultDivOp>} &::= * \mid / \\
\texttt{<Body>} &::= \texttt{<StmtList>} \mid \texttt{<Return>} \mid \texttt{<StmtList>} \mathbin{+\!\!+} \texttt{<Return>} \\
\texttt{<StmtList>} &::= \texttt{<Statement>} \mid \texttt{<StmtList>} \mathbin{+\!\!+} \texttt{<Statement>} \\
\texttt{<Statement>} &::= \texttt{<Declaration>;} \mid \texttt{<Assignment>;} \\
\texttt{<Declaration>} &::= \texttt{var <Var>} \mid \texttt{var <Var> = <Exp>} \\
\texttt{<Assignment>} &::= \texttt{<Var> = <Exp>} \\
\texttt{<Return>} &::= \texttt{<Exp>} \\
\texttt{<Call>} &::= \texttt{<Var> = <Exp>} \\
\texttt{<Fn>} &::= \\
\texttt{<FnSig>} &::= \texttt{<Var>()} \\
\texttt{<Letter>} &::= [a - zA - Z] \\
\texttt{<Var>} &::= \texttt{<Letter>} \mid \texttt{<Var>} \mathbin{+\!\!+} \texttt{<Letter>} \mid \texttt{<Var>} \mathbin{+\!\!+} \texttt{<Digit>}
\end{aligned}
$$